

PFE

Primenjena Fizika i Elektronika
Zimski kamp za starije polaznike

Vežba

Sinteza

N. Petrović



Mart 2026

Vežve su bazirane na: <https://pulp-platform.org/efclwinter2026/>

1 Uvod

Sinteza je postupak prevođenja opisa digitalnog kola, napisanog na hardverskom jeziku kao što su **SystemVerilog** ili **VHDL**, u strukturu logičkih elemenata koji mogu da se realizuju u IC tehnologiji. Ulaz u proces sinteze predstavlja **RTL opis** kola, gde su definisani registri, kombi-nacionala logika i ponašanje sistema na nivou prenosa između registara.

Tokom sinteze, alat analizira opis kola i mapira ga na elemente iz odgovarajuće biblioteke standardnih ćelija, kao što su na primer multiplekseri, flip-flopovi, SRAM ćelije i slično. Pri tome se vodi računa o zadatim ograničenjima, pre svega o **radnom taktu**, ali i o zauzeću površine i potrošnji energije. Rezultat sinteze je **gate-level netlista**, odnosno opis kola na nivou logičkih ćelija (engl. *logic cells*) i makroa (engl. *macro*), koja se dalje koristi u narednim fazama projektovanja čipa.

Značaj sinteze je u tome što predstavlja vezu između apstraktnog opisa hardvera i njegove stvarne fizičke realizacije. Na taj način omogućava proveru da li projektovano kolo može da ispuni zadate tehničke zahteve pre prelaska na složenije faze kao što su *place-and-route*, analiza kašnjenja i fizička implementacija.

Cilj ove laboratorijske vežbe je da upoznate postupak sinteze digitalnog kola, kao i da analizirate rezultate sinteze, poput iskorišćene logike, procenjenog kašnjenja i eventualnih upozorenja koje alat generiše.

Ćelije i makroi se nalaze u kolekciji nazvanoj biblioteka standardnih ćelija (engl. *standard cell library*). Biblioteka zajedno sa svim tehnološki specifičnim podacima čini process design kit (PDK). Za potrebe ove vežbe koristićemo IHP130 PDK¹ otvorenog koda.

1.1 O stilu

Zadatak za studente:

Delovi teksta sa sivom pozadinom označavaju korake potrebne za završavanje vežbe.

Tokom vežbe unosite komande putem komandne linije².

1.2 Početak rada

Zadatak 1:

- Uđite u direktorijuma vežbe

```
sh > cd ~/Desktop/Projects/pfe_mart_2026
```

- Pokrenite docker

```
sh > ./run_docker.sh
```

- Prebacite se u folder za sintezu

```
sh > cd exercises/asic/yosys
```

¹<https://github.com/IHP-GmbH/IHP-Open-PDK>

²Komandna linija omogućava skriptovanje i ponovljivost.

Napomena 1: Ako otvorite novi terminal, ne zaboravite da ponovo pokrenete docker.

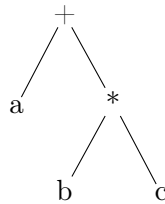
Napomena 2: Ova vežba se radi u folderu yosys.

2 Sinteza: Pregled

Pre same sinteze pomoću Yosys alata, pogledajmo korake koje apstraktni alat za sintezu mora da obavi.

1. Parsiranje

- Prvi korak je učitavanje bihevioralnog opisa, u našem slučaju SystemVerilog koda, u alat za sintezu. Alat za sintezu mora da parsira osobine HDL jezika i da ih prevede u internu reprezentaciju. Obično to najpre podrazumeva parsiranje dizajna u apstraktno sintaksko stablo (engl. *Abstract Syntax Tree - AST*), reprezentaciju koja veoma podseća na način na koji je HDL jezik strukturno organizovan. Zatim se u drugom koraku AST prevodi u reprezentaciju pogodniju za sintezu. U toj reprezentaciji više nije važno kako je neka operacija implementirana, već šta ona radi i kako je povezana sa drugim operacijama. U tom trenutku dizajn je i dalje bihevioralan, odnosno sadrži osobine nalik kodu koje omogućavaju da se nešto implementira na apstraktan način.
- Primer: $a + b * c$



2. Elaboracija

- Sledeći korak je da se bihevioralni dizajn prevede u strukturni dizajn. Dok bihevioralni model može da sadrži apstraktne konstrukcije kao što su petlje ili uslovna grananja, strukturni model je veoma blizak netlisti. On sadrži samo ćelije (aritmetičke operacije, mašine konačnih stanja i slično) i veze između tih ćelija. Svi parametri u dizajnu, na primer širina magistrale podataka, mogu se postaviti na svoje konačne vrednosti, a odgovarajuće vrednosti će tokom elaboracije biti propagirane kroz poddizajne.

3. Logička optimizacija

- Primenom različitih tehnika, strukturna reprezentacija dizajna može se optimizovati. To znači da se smanjuje dužina najdužih putanja, uklanjaju se redundantne operacije i deli se izlaz zajedničkih operacija, propagiraju se konstantni bitovi i slično.

4. Mapiranje

- Sa koracima logičke optimizacije prepliću se i koraci mapiranja. U mapiranju se jedna ili više ćelija prevode u drugu reprezentaciju. U praksi to obično znači da se polazi od ćelija koje predstavljaju operacije veoma visokog nivoa, kao što je proizvoljna aritmetička operacija, a zatim se pomoću mapiranja one transformišu u reprezentaciju nižeg nivoa, na primer u logička kola. U završnim fazama sinteze interna reprezentacija se mora mapirati na standardne ćelije koje obezbeđuje PDK; taj postupak se naziva tehnološko mapiranje.

U komercijalnim alatima takođe se zadaje skup ograničenja koja opisuju vremenske zahteve koje dizajn mora da ispuni, na primer različiti delovi sistema mogu koristiti različite periode takta. Tokom celog procesa alat nastoji da ostvari postavljene ciljeve. Pokušava da ispuni data vremenska ograničenja, a na svim nekritičnim putanjama može zatim da smanji zauzeće logičke površine pronalaženjem kompaktnijih reprezentacija iste funkcije, što obično znači da su one sporije.

Trenutno alati otvorenog koda ne podržavaju sisteme sa više različitih taktnih signala.



Šta da se radi. Više sreće sledeći put.

3 Sinteza u Yosys-u

U prethodnom odeljku naučili ste koji su koraci neophodni za sintezu dizajna opisanog u jeziku SystemVerilog. Sada ćemo korak po korak proći kroz proces sinteze koristeći alate Yosys i yosys-slang. Celokupan tok, od ulaznog opisa do elaboriranog dizajna, izgleda ovako:

- yosys-slang
 - Parsira izvorne fajlove, razrešava direktive, gradi stablo modula, unikatira instance i elaborira module u Yosys-ove interne ćelije.
- Yosys
 - Elaborirani dizajn je dostupan u Yosys-ovoj internoj reprezentaciji.

Napomena 3: Ova vežba se radi u folderu `yosys`. Tokom vežbe izvršićete mnogo komandi. Preporučujemo da kreirate sopstvenu skriptu. Yosys čita `.ys` skripte (komandom `script`) i `.tcl` skripte (komandom `tcl`).

Napomena 4: Korake ove vežbe je potrebno da upisujete i u termina i u praznu `tcl` skriptu koja je pripremljena za vas i nalazi se u `pfe/sinteza.tcl`. Ne zaboravite, nalazimo se u `yosys` folderu.

Za početak, pokrenimo prvo potpuni tok sinteze za jednostavno kolo. Pokrenuti (unutar `yosys` foldera):

```
sh > ./run_synthesis.sh --logic
```

Ova komanda pokreće ceo postupak sinteze i kroz ovaj postupak ćemo malo kasnije proći zajedno.

Po završetku sinteze pogledati fajl `out/logic_yosys.v`. RTL kod je dat `../rtl/pfe_soc_logic.sv`. Kako se dobijena netlista razlikuje od početnog RTL koda?

Prodiskutovati.

3.1 Učitavanje tehnoloških biblioteka

Prvo je potrebno učitati tehnološku biblioteku, koja je od suštinskog značaja za tok sinteze. Tehnološka biblioteka definiše karakteristike standardnih ćelija, uključujući njihova električna svojstva, vremenske karakteristike i modele potrošnje energije. Ovi podaci su ključni kako bi se obezbedilo da dizajn zadovolji ograničenja u pogledu vremenskih zahteva, površine i potrošnje tokom sinteze, kao i u daljem toku projektovanja integrisanih kola. U ovoj vežbi koristićemo standardnu biblioteku ćelija IHP-SG13G2, zasnovane na CMOS procesu od 130 nm.

Biblioteka IHP-SG13G2 sastoji se iz tri komponente:

- **Biblioteka standardnih ćelija:** Sadrži logička kola, flip-flobove i druge osnovne elemente koji se koriste za izgradnju jezgra digitalnog dizajna.
- **SRAM biblioteka:** Obezbeđuje memorijske blokove kao što je SRAM, koji se često koriste za skladištenje podataka unutar dizajna.
- **I/O biblioteka:** Sadrži ulazno/izlazne padove za povezivanje dizajna sa spoljnim komponentama.

Datoteke tehnološke biblioteke možete direktno pregledati, pošto su sačuvane u čoveku čitljivom .lib formatu, odlaskom u direktorijum tehnologije. Ove datoteke nalaze se u direktorijumu `../ihp13/pdk/ihp-sg13g2/libs.ref`.

Nazivi datoteka biblioteke standardnih ćelija prate sledeći format:

`<naziv>_<tip_celije>_<proces>_<napon>_<temperatura>.lib`

Na primer, naziv datoteke

`sg13g2_stdcell_typ_1p20V_25C.lib`

može se rastaviti na sledeći način:

- `sg13g2`: naziv biblioteke.
- `stdcell`: ćelije u ovoj biblioteci su standardne ćelije
- `typ`: označava tipični proces.
- `1p20v`: nominalni napon od 1.2 V.
- `25c`: temperatura karakterizacije od 25 °C.

Otvorimo datoteku `sg13g2_stdcell_typ_1p20V_25C.lib` koja se nalazi u sledećem folderu: `../ihp13/pdk/ihp-sg13g2/libs.ref/sg13g2_stdcell/lib/`. Pogledajmo šta se nalazi u njoj. Na početku, datoteka navodi neke globalne informacije, kao što su podrazumevane jedinice, podrazumevani parametri, modeli otpornosti i kapacitivnosti provodnika, modeli potrošnje energije i slično. Nakon toga, biblioteka definiše karakteristike za svaku standardnu ćeliju u tehnologiji. Uzmimo nand kolo `sg13g2_nand2_1` kao primer. Biblioteka prvo navodi površinu ćelije i njenu statičku potrošnju usled curenja. Zatim opisuje karakteristike izlaznog pina Y. Takojd, možete videti funkciju pina Y definisanu kao:

```
function :  "!(A*B)";.
```

Ova informacija je potrebna alatu za sintezu kako bi pravilno odabrao standardne ćelije za implementaciju logičke funkcije. Nakon toga videćete više tabela koje opisuju vremensko ponašanje i potrošnju ćelije. Ove informacije koriste se da bi se zadovoljili vremenski zahtevi u fazi sinteze. Alat za sintezu koristi te podatke da generiše netlistu koja implementira logiku i pritom zadovoljava postavljena ograničenja.

U ovoj vežbi koristićemo tipični ugao pri 1.2 V i 25 °C, predstavljen datotekama sa sufiksom `typ_1p20v_25c` za biblioteku standardnih ćelija, `*_typ_1p20V_25C` za SG130 biblioteku SRAM makroa i `*_typ_1p2V_3p3V_25C` za SG130 biblioteku I/O ćelija. Ovaj ugao predstavlja nominalne uslove, koji se obično koriste za početnu sintezu i vremensku analizu. Kasnije u procesu dizajna možete istražiti i druge uglove (na primer spori ili brzi ugao) kako biste sproveli sveobuhvatnu proveru vremenskih karakteristika u najgorim ili najboljim uslovima rada.

Kada identifikujete tehnološke biblioteke koje se koriste u vašem dizajnu i pronađete njihove putanje, sledeći korak je da te biblioteke učitate u Yosys. To omogućava alatu za sintezu da koristi standardne ćelije, SRAM-ove i I/O padove za mapiranje i sintezu vašeg dizajna.

Napomena 5: U okviru Yosys programa koristite `help` za više informacija o komandi.

Napomena 6: Ukoliko se suprotno ne naglasi, komande unosite u yosys komandnu liniju i zapisujete ih u skriptu `pfe/sinteza.tcl`.

Zadatak 2: Učitavanje tehnoloških biblioteka

- U terminalu koji ste inicijalno otvorili, ukoliko se nalazite u yosys folderu, pokrenuti:

```
sh > rm -r tmp/ out/* reports/* *.log # Ovo briše generisane fajlove
sh > yosys -C # Komanda pokreće yosys u interaktivnom modu
```

- Tehnološki podaci se učitavaju pomoću komande (ne upisivati u komandnu liniju, ovo je samo objašnjenje):

```
yosys > yosys read_liberty -lib <path_to_library_file>
```

Uglaste zagrade `< >` označavaju da trebate uneti sami nešto umesto njih.

- Medjutim, kako ovih fajlova ima mnogo, možete ih sve dodati unutar yosys komandne linije pomoću:

```
set pdk_dir "../ihp13/pdk"
set pdk_cells_lib ${pdk_dir}/ihp-sg13g2/libs.ref/sg13g2_stdcell/lib
set pdk_sram_lib  ${pdk_dir}/ihp-sg13g2/libs.ref/sg13g2_sram/lib
set pdk_io_lib    ${pdk_dir}/ihp-sg13g2/libs.ref/sg13g2_io/lib

set tech_cells [list "$pdk_cells_lib/sg13g2_stdcell_typ_1p20V_25C.lib"]
set tech_macros [glob -directory $pdk_sram_lib *_typ_1p20V_25C.lib]
lappend tech_macros "$pdk_io_lib/sg13g2_io_typ_1p2V_3p3V_25C.lib"

# svi lib fajlovi
set lib_list [concat [split $tech_cells] [split $tech_macros] ]
set liberty_args_list [lmap lib $lib_list {concat "-liberty" $lib}]
set liberty_args [concat {*}$liberty_args_list]
# only the standard cells
set tech_cells_args_list [lmap lib $tech_cells {concat "-liberty" $lib}]
set tech_cells_args [concat {*}$tech_cells_args_list]
```

```
# procitaj library datoteke
foreach file $lib_list {
    yosys read_liberty -lib "$file"
}
```

4 Učitavanje dizajna

Sledeći korak u toku sinteze jeste učitavanje datoteka dizajna u Yosys. Yosys je sposoban da čita i obrađuje Verilog-2005 kod, a SystemVerilog dizajne može da obradi pomoću dodatka `yosys-slang`.

Zadatak 3: Učitavanje dizajna

- Učitajte dizajn korišćenjem `yosys-slang` plugina:

```
yosys> plugin -i slang.so
```

Možete koristiti komandu `help read_slang` u okviru Yosys-a kako biste saznali više o dostupnim opcijama i načinu podešavanja ove komande (ovo funkcioniše samo ako je dodatak prethodno omogućen).

- Nakon što je dodatak omogućen, možemo koristiti komandu `read_slang`, koja nam omogućava učitavanje SystemVerilog datoteka.

```
yosys> yosys read_slang --no-proc --top pfe_chip \
      -f ./src/pfe.flist --keep-hierarchy
```

Ova komanda određuje hijerarhijski najviši modul dizajna i navodi listu izvornih datoteka koje čine dizajn. Opcija `-top` definiše ime najvišeg modula, dok se opcija `-F` koristi za zadavanje liste datoteka koja sadrži sve potrebne fajlove dizajna. Obratite pažnju da u našem slučaju koristimo opciju `-f` umesto `-F`, kako bismo obezbedili da se relativne putanje izvornih datoteka tumače počev od trenutnog direktorijuma (odnosno direktorijuma `yosys` u našem slučaju). Za ovu vežbu možete pronaći unapred generisanu listu datoteka za naš dizajn u `yosys/src/pfe.flist`. Ime najvišeg modula je `pfe_chip`.

Napomena 7: Imajte u vidu da ćete za vaš projekat najverovatnije morati da menjate `*.flist` fajl.

Nakon što je dizajn parsiran pomoću alata Slang, moguće je ispisati informacije o učitanoj dizajnu komandom `stat`. Komanda `write_verilog` koristi se za čuvanje trenutnog dizajna kao Verilog netliste. Dokumentaciju ovih komandi možete pogledati za više informacija. Da bismo vam olakšali početak, u nastavku su dati neki primeri kako se ove komande koriste:

```
# ispisivanje u komandnoj liniji
yosys > yosys stat

# generisati izveštaj 'pfe_parsed.rpt' u reports folderu
yosys > yosys tee -q -o "reports/pfe_parsed.rpt" stat

# generisati netlistu
yosys > yosys write_verilog -norename -noexpr "out/pfe_parsed.v"
```

- Pre nego što nastavimo dalje, potrebno je da damo par naredbi alatu. Potrebno je da

kažemo alatu za koje module zelimo da sačuvamo hijerarhiju kao i to da imamo SRAM ćeliju u dizajnu:

```
# Ne morate sacuvati sve hijerarhije, ali sacuvajte makar akumulator
yosys > yosys setattr -set keep_hierarchy 1 "t:pfe_soc$"
yosys > yosys setattr -set keep_hierarchy 1 "t:accumulator$"

yosys > yosys blackbox "t:RM_IHPSG13_2P_256x8_c2_bm_bist$"
```

U ovom primeru, sačuvali smo hijerarhiju `pfe_soc` i `accumulator` modula.

SRAM ćelija koju koristimo u dizajnu je `RM_IHPSG13_2P_256x8_c2_bm_bist` pa smo morali da je dodamo kao `blackbox`.

- Takodje, kako u `pfe_chip` modulu imamo pad-ove za napajanje koji naizgled deluju nepovezano, potrebno je da dodamo sledeće komande kako ih `yosys` ne bi optimizovao:

```
yosys > yosys attrmap -rename dont_touch keep
yosys > yosys attrmap -tocase keep -imap keep="true" keep=1
yosys > yosys attrmvp -copy -attr keep
```

Napomena 8: Kod kopiran iz `pfe_chip` modula:

```
(* dont_touch = "true" *)sg13g2_IOPadIOVss pad_vssio0();
(* dont_touch = "true" *)sg13g2_IOPadIOVss pad_vssio1();
(* dont_touch = "true" *)sg13g2_IOPadIOVss pad_vssio2();
(* dont_touch = "true" *)sg13g2_IOPadIOVss pad_vssio3();
```

5 Elaboracija dizajna

Zadatak 4: Elaboracija

- Elaboracija transformiše HDL dizajn u međureprezentaciju, pri čemu se "razrešava" hijerarhija dizajna i instanciraju svi moduli. Ovaj korak obezbeđuje da su sve komponente na svom mestu pre nego što optimizacija može da započne. Tokom faze elaboracije koriste se sledeće komande:

```
yosys > yosys hierarchy -top <top_module>
yosys > yosys check
yosys > yosys proc
```

Pri čemu `<top_module>` treba zameniti sa našim glavnim modulom.

- Generisati izveštaje

```
yosys> yosys tee -q -o "reports/pfe_elaborated.rpt" stat
yosys> yosys write_verilog -norenname -noexpr -attr2comment out/pfe_elaborated.v
```

Proveriti kako u `pfe_elaborated.v` izgleda akumulator (`accumulator`).

5.1 Optimizacija visokog nivoa

Možda ste primetili da sve ćelije koje se trenutno nalaze u našem dizajnu počinju znakom \$, a da su njihova imena napisana malim slovima. To je karakteristično za reprezentaciju visokog nivoa (odnosno apstraktnu reprezentaciju) u Yosys-u. Naziva se reprezentacijom visokog nivoa zato što jedna ćelija može da predstavlja veliku i složenu operaciju, kao što je deljenje ili množenje.

Najčešće optimizacione naredbe u Yosys-u počinju sa `opt_*`. Postoji i šira `opt` prolazna naredba (*pass*) koja izvršava niz `opt_*` potkomandi. Četiri najvažnija načina pozivanja naredbe `opt` su:

- `opt`: prolaz sa podrazumevanim podešavanjima
- `opt -noff`: isto kao prethodno, ali bez optimizacija koje se odnose na flip-flopove (integracija *enable* signala, uklanjanje konstanti i slično)
- `opt -fast`: izvršava alternativni niz potkomandi koji možda ne daje baš najbolje rezultate, ali traje kraće
- `opt -full`: uključuje i neke dodatne optimizacije
- `fsm`: koristi se za izdvajanje i optimizaciju mašina konačnih stanja (*Finite State Machines*, FSM) iz dizajna.

U Yosys-u, gruba sinteza (*coarse-grain synthesis*) predstavlja fazu u kojoj je dizajn još uvek opisan na višem nivou apstrakcije. U ovoj fazi logičke operacije izražene su pomoću operatora na nivou reči, koji obrađuju viševitne podatke (na primer reči od 16, 32 ili 64 bita) umesto pojedinačnih bitova. Ovi operatori mogu obuhvatati funkcionalne jedinice kao što su sabirači, množači ili pomerači, koji obrađuju blokove podataka na apstraktniji način. Na primer, 32-bitni sabirač je i dalje prepoznatljiv kao sabirač, ali njegova unutrašnja struktura još nije razložena na pojedinačna logička kola kao što su AND, OR i NOT.

Tokom ove faze, Yosys obrađuje važne komponente kao što su mašine konačnih stanja (FSM) i memorije, prevodeći ih iz opisa ponašanja višeg nivoa u reprezentacije pogodne za hardversku implementaciju. Na primer, FSM-ovi se kodiraju u registre stanja i upravljačku logiku, dok se memorije mapiraju na tehnološki specifične RAM blokove ili se sintetišu korišćenjem nizova registara.

Redosled transformacija i optimizacija u ovoj fazi igra ključnu ulogu u obezbeđivanju optimalnih performansi i efikasnog korišćenja resursa. Za većinu koraka opisanih u narednom odeljku postoji po jedna komanda koja izvršava sve opisane transformacije. Međutim, većina tih komandi su zapravo makroi koji se, prilikom izvršavanja u Yosys komandnoj liniji, proširuju u više pojedinačnih komandi. U nastavku ćemo proći kroz ključne korake i komande koji se koriste u gruboj sintezi.

Zadatak 5: Gruba sinteza

- Najpre, pokrenimo komandu `check` koja služi za rano prepoznavanje potencijalnih problema u dizajnu tokom toka sinteze. Pokretanjem ove komande možemo uočiti greške pre nego što nastavimo dalje, čime se štedi vreme i sprečavaju složeniji problemi u kasnijim fazama procesa:

```
yosys > yosys check
```

- pokrenimo optimizaciju koja isključuje flip-flopove i optimizaciju FSM:

```
yosys > yosys opt -noff
```

```
yosys > yosys fsm
```

- Generisati izveštaje i proveriti generisane fajlove:

```
yosys > yosys write_verilog -norename -noexpr out/pfe_postfsm.v  
yosys > yosys tee -q -o "reports/pfe_postfsm.rpt" stat -width -tech cmos
```

- Pokrenuti naredbe **wreduce** koja smanjuje širinu operacija (odnosno broj bitova) i **peepopt** koja izvršava određene optimizacije specifične za operatore. Na kraju je potrebno očistiti dizajn od komponenti koje se ne koriste pomoću **opt_clean** komande i pokrenuti opet optimizaciju:

```
yosys > yosys wreduce  
yosys > yosys peepopt  
yosys > yosys opt_clean  
yosys > yosys opt -full
```

- Komanda **share** objedinjuje resurse koji se mogu deliti u jednu instancu i pokušava da utvrdi da li određeni resursi mogu da dele istu hardversku implementaciju. Ova optimizacija je korisna za smanjenje redundantnih resursa, kao što su duplirane aritmetičke jedinice, u sintetisanoj netlisti.

```
yosys > yosys share  
yosys > yosys opt
```

- U HDL projektovanju, memorijski blokovi kao što su SRAM i ROM mogu biti ili eksplicitno instancirani kao makro blokovi, ili ih alat može izvesti iz bihejvioralnog opisa dizajna. Kod ASIC projektovanja standardna praksa je eksplicitna instancijacija prethodno generisanih memorijskih makroa (imamo SRAM ćelije u dizajnu). Komanda **memory** se koristi za generisanje optimizovanih dekodera adresa i registara za velike nizove flip-flopova.
- Izvršiti izvođenje memorija i optimizaciju registarskog fajla pomoću komande **memory**. Nakon toga pokrenite optimizaciju sa opcijom **-fast**.
- Optimizujte flip-flopove u dizajnu pomoću komande **opt_dff**. Koristite opciju **-sat** kako biste uklonili flip-flopove čiji se ulazi mogu dokazati kao konstantni pomoću SAT rešavača. Koristite opcije **-nodffe** i **-nosdff** da onemogućite transformacije **dff -> dffe** sa signalom omogućavanja i **dff -> sdff** sa sinhronim reset/set signalom.
- Ponovo pokrenite kompletnu optimizaciju. Očistite dizajn pomoću komande **clean**; koristite opciju **-purge** za uklanjanje internih mreža sa javnim imenima.
- Generišite i analizirajte izveštaj. Šta se promenilo nakon optimizacije flip-flopova?

```
yosys > yosys memory -nomap  
yosys > yosys memory_map  
yosys > yosys opt -fast  
  
yosys > yosys opt_dff -sat -nodffe -nosdff  
yosys > yosys opt -full  
yosys > yosys clean -purge  
  
yosys > yosys tee -q -o "reports/pfe_abstract.rpt" stat -width -tech cmos  
yosys > yosys write_verilog -norename -noexpr "out/pfe_abstract.v"
```

6 Ograničenja

U digitalnom projektovanju, skup specifikacija, kao što su performanse, površina i potrošnja energije, mora biti zadovoljen da bi dizajn bio ispravan. Ove specifikacije nazivaju se ograničenjima i moraju biti saopštene alatu za sintezu pre početka procesa kompajliranja.

Ograničenja definišu vaše “zahteve” prema alatu za sintezu. Alat će pokušati da vaš HDL opis prevede u gate-level netlistu koja zadovoljava ova ograničenja. Ipak, važno je napomenuti da, iako će alat nastojati da ispunji ove zahteve, ne postoji garancija da će sva ograničenja biti u potpunosti zadovoljena. Nakon sinteze, neophodno je proveriti da li dizajn zaista ispunjava željena ograničenja, jer alat to možda neće moći da postigne usled ograničenja resursa ili međusobno suprotstavljenih specifikacija.

6.1 Definisanje perioda takta

Jedno od najvažnijih ograničenja jeste frekvencija na kojoj je predviđeno da kolo radi. Period takta određuje koliko brzo dizajn može da funkcioniše, pa je zato od suštinskog značaja da se ovo ograničenje zada u ranoj fazi projektovanja. U ovom odeljku postaviti ćemo ciljnu period takta za naš primer dizajna.

Zadatak 6: Definisanje taktnog signala

- Napravite promenljivu za period takta. U Yosys-u se ciljno kašnjenje (željeni period takta) podrazumevano zadaje u pikosekundama (za više detalja pogledajte Yosys ABC dokumentaciju). Da biste definisali period takta od 20 000 pikosekundi (što odgovara frekvenciji od 50 MHz), koristite sledeću komandu:

```
yosys > set period_ps 20000
```

Imajte u vidu da, iako vas u ovom koraku tražimo da definišete TCL promenljivu zato što trenutno govorimo o ograničenjima, ne postoji poseban razlog da se promenljiva baš zove `period_ps` (niti je promenljiva uopšte strogo neophodna). Iako je iz nastavnih razloga zgodno da se sva ograničenja definišu u istom trenutku, treba imati na umu da je jedino što je zaista potrebno za zadavanje vremenskih ograničenja prosleđivanje perioda takta kao posebne opcije komandne linije komandi `abc`, kao što ćete videti malo kasnije.

6.2 Podešavanje ulaznih drajvera i izlaznog opterećenja

U praksi, ulazni signali nisu idealni i njihove ivice imaju konačna vremena porasta i opadanja usled pogonske snage kola koja ih generišu. Jačina ulaznog drajvera utiče na vremenske karakteristike i ukupne performanse dizajna. U ovom odeljku definisaćemo drajverske ćelije za ulazne signale i zadati izlazno opterećenje koje će dizajn pobuđivati.

Drajverska ćelija definiše jačinu ulaznog signala. Zadavanjem drajverske ćelije alatu za sintezu saopštavate koliko su ulazni signali jaki ili slabi, što mu omogućava da na odgovarajući način prilagodi dizajn. U našem primeru možemo koristiti bafer ćelije (`BUFX2`) kao referentne ulazne drajverske ćelije.

Numerički sufiks `x` označava veličinu ćelije i odražava njenu relativnu pogonsku snagu. Jači bafer (na primer `BUFX4`) pobuđuje opterećenje brže. Međutim, korišćenje prejakog bafera možda neće verno predstavljati stvarno ponašanje čipa, odnosno možda neće odgovarati realnim uslovima rada. U našem primeru drajversku ćeliju treba postaviti na `BUFX2`.

U kasnijim fazama projektovanja potrebna je datoteka sa ograničenjima kako bi se sva ograničenja prosledila alatu Yosys. Obezbedili smo šablon za ABC datoteku sa ograničenjima u datoteci `/src/abc.constr`. Sledećom komandom možete zadati ulaznu drajversku ćeliju:

```
> set_driving_cell <cell>
```

Kašnjenje propagacije u CMOS kolima u velikoj meri zavisi od kapacitivnog opterećenja koje izlaz pobuđuje. Veća kapacitivnost opterećenja znači da je potrebno više vremena da se izlaz napuni ili isprazni, što direktno povećava kašnjenje i utiče na brzinu propagacije signala. Korišćenje ulazne kapacitivnosti bafer ćelije kao aproksimacije opterećenja uobičajena je praksa, jer pouzdano predstavlja efektivno kapacitivno opterećenje koje vidi drajverska ćelija. Na primer, u datoteci `sg13g2_stdcell_typ_1p20V_25C.lib` možemo pronaći da je ulazna kapacitivnost pina ćelije `sg13g2_buf_2` jednaka `0.00261926 pF`.

U mnogim slučajevima projektanti biraju veću vrednost kapacitivnosti kako bi obezbedili da vremenska ograničenja budu zadovoljena čak i u uslovima najnepovoljnijeg opterećenja.

Sledećom komandom možete zadati izlazno opterećenje:

```
> set_load <load>
```

Zadatak 7: Podešavanje ulazno/izlaznih opterećenja

- Modifikovati fajl `src/pfe.constr` i uneti:

```
set_driving_cell sg13g2_buf_2
set_load 0.003
```

Sačuvati ovaj fajl.

7 Tehnološko mapiranje

Do sada smo videli kako se HDL kod transformiše u RTL netlistu. RTL netlista je i dalje opisana pomoću apstraktnih ćelija grubog nivoa, kao što su široki sabirači, množači i memorijski elementi. Sada ćemo razmotriti kako se ova RTL netlista transformiše u funkcionalno ekvivalentnu netlistu sastavljenu od instanci standardnih ćelija iz ciljne tehnološke biblioteke i žica koje ih povezuju, čime postaje spremna za fizičku implementaciju.

Tehnološko mapiranje se obično sastoji iz dve ključne faze:

- **Zamena ćelija:** U ovoj fazi RTL ćelije se mapiraju na internu Yosys biblioteku generičkih (odnosno od tehnologije nezavisnih) jednobitnih gate-level ćelija. Ovo služi kao međukorak pre mapiranja na stvarne tehnološki specifične ćelije.
- **Gate-level tehnološko mapiranje:** U ovoj fazi netlista sastavljena od jednobitnih logičkih kola transformiše se u netlistu koja koristi logička kola iz ciljne tehnološke biblioteke. Ovaj korak obezbeđuje da se dizajn sastoji od ćelija dostupnih u konkretnom proizvodnom procesu.

Cilj tehnološkog mapiranja jeste da se obezbedi da sintetisani dizajn može biti implementiran korišćenjem dostupne tehnologije, uz optimizaciju faktora kao što su vremenske karakteristike i površina.

7.1 Zamena ćelija

Zamena ćelija obavlja se pomoću komande `techmap`. Ovaj korak zamenjuje RTL ćelije jednostavnijim ćelijama nižeg nivoa, bilo iz prosledene Verilog datoteke ili iz ugrađenih Yosys mapi-

ranja. Podrazumevano, ako nije navedena posebna mapirajuća datoteka, prolaz **techmap** koristi ugrađenu mapirajuću datoteku za zamenu Yosys RTL tipova ćelija internim gate-level ćelijama.

Zadatak 8: Generičko mapiranje

- Pokrenite prolaz **techmap** kako biste zamenili RTL ćelije u dizajnu internim gate-level ćelijama alata Yosys. Nakon mapiranja pokrenite komande **opt** i **clean**.
- Generišite i analizirajte izveštaj. Posmatrajući izveštaj, možete li da zaključite šta se promenilo?

```
yosys > yosys techmap
yosys > yosys opt
yosys > yosys clean -purge

yosys > yosys tee -q -o "reports/pfe_generic.rpt" stat -tech cmos
yosys > yosys write_verilog -norename -noexpr "out/pfe_generic.v"
```

Sada ponovo pronađite **accumulator** i posmatrajte šta se promenilo.

7.2 Gate-level tehnološko mapiranje

Na gate-level nivou, ciljna tehnologija opisana je Liberty datotekama (format **.lib**), koje sadrže informacije o dostupnim ćelijama, kao i o njihovim vremenskim karakteristikama i potrošnji energije. Liberty datoteke se učitavaju na početku toka sinteze, kao što je to već pomenuto. Tehnološko mapiranje u ovoj fazi obavlja se kroz dve ključne etape.

Najpre se registarske ćelije mapiraju na registre dostupne u ciljnoj tehnologiji pomoću prolaza **dfflibmap**. Ovaj prolaz očekuje Liberty datoteku kao argument (pomoću opcije **-liberty**) i koristi isključivo registarske ćelije iz te datoteke.

Nakon toga, kombinaciona logika se mapira na ćelije ciljne tehnologije. Za ovaj korak Yosys se oslanja na spoljašnji alat ABC (putem komande **abc**). ABC takođe koristi Liberty datoteku kao ulaz, ali se fokusira na kombinacione ćelije. Pored toga, ABC omogućava zadavanje ograničenja, kao što su željeni period takta (ciljno kašnjenje), pobudna ćelija i izlazno opterećenje.

Radi jednostavnosti, obezbeđujemo vam prilagođenu ABC optimizacionu skriptu koju možete naći ovde: **/scripts/abc-opt.script**. Ova skripta automatizuje iterativni tok optimizacije i mapiranja. Skripta najpre učitava dizajn i primenjuje više krugova intenzivne optimizacije kašnjenja, koristeći strukturno heširanje (**&sh**), prepisivanje (**&if**), sintezu (**&syn2**) i balansiranje (**&b**), kako bi se unapredila logička struktura kola. Zatim se smenjuju iteracije optimizacije (**&opt_iter**) i iteracije mapiranja (**&map_iter**), pri čemu se logika bira i mapira na ciljnu tehnologiju, uz kontinuirano ažuriranje statistike dizajna (**&ps**). Na kraju, skripta izvršava topološko sortiranje (**topo**), vremensku analizu (**stime**) i fizička prilagođavanja, kao što su ubacivanje bafera (**buffer**) i promena veličine ćelija, kako bi se dodatno optimizovali kašnjenje i fanout, što rezultuje dobro optimizovanom netlistom prilagođenom vremenskim zahtevima. Za sopstveni projekat možda ćete želeći da razvijete optimizacionu skriptu koja odgovara specifičnostima vašeg dizajna ali ovo je van domena ovog kursa.

Zadatak 9: Tehnološko mapiranje

- Pokrenuti:

```
# Poravnanje hijerarhije
yosys > yosys flatten
```

```
# Mapiranje flip-flopova
yosys > yosys dfflibmap {*} $tech_cells_args

# Podesavanje abc optimizacije
yosys > set abc_comb_script "scripts/abc-opt.script"

# ABC optimizacija
yosys > yosys abc {*} $tech_cells_args -D $period_ps \
        -script $abc_comb_script -constr src/pfe.constr {*}[list] -showtmp

yosys > yosys clean -purge
```

8 Priprema za OpenROAD

Ovo je poslednji korak u toku sinteze! Do sada smo uspešno transformisali HDL kod u gate-level netlistu i mapirali je na ciljnu tehnološku biblioteku. Sada ćemo pripremiti netlistu za fazu fizičkog projektovanja pomoću alata OpenROAD. Sledeće komande koriste se kako bi izlazna netlista bila usklađena sa zahtevima OpenROAD-a:

- **setundef** – zamenjuje nedefinisane konstante (x) definisanim konstantama (0/1).
- **splitnets** – deli višebitne mreže na jednobitne mreže.
- **hilomap** – mapira konstante na drajverske ćelije **tielo** i **tiehi**.

8.1 Razdvajanje višebitnih mreža (magistrala)

Višebitne mreže predstavljaju grupe signala koje prenose više bitova podataka (na primer magistrale). Iako su višebitne mreže pogodnije za opise na višem nivou, alati za fizičko projektovanje kao što je OpenROAD očekuju da netlista bude predstavljena na detaljnijem nivou, odnosno kroz jednobitne mreže. Razdvajanjem višebitnih mreža na jednobitne činimo netlistu kompatibilnijom sa ovakvim alatima, obezbeđujući da se svaki vod može zasebno rutirati i optimizovati tokom faze postavljanja i rutiranja. Time se takođe izbegava dvosmislenost u fizičkom projektovanju prilikom rada sa magistralama.

8.2 Zamena nedefinisanih konstanti

U digitalnom projektovanju, nedefinisane konstante (stanja x) predstavljaju nepoznatu ili neodređenu vrednost. Iako se takve vrednosti mogu koristiti u simulaciji ili verifikaciji radi otkrivanja potencijalnih grešaka, alati za fizičko projektovanje kao što je OpenROAD zahtevaju determinističke vrednosti (ili 0 ili 1) za implementaciju. Nedefinisane vrednosti nije moguće fizički realizovati tranzistorima, pa njihovom zamenom definisanim konstantama obezbeđujemo da netlista bude fizički ostvariva i da ne postoje neodređenosti koje bi mogle dovesti do problema tokom sinteze ili rutiranja.

Zadatak 10: Priprema za OpenROAD

- Pokrenuti:

```
# Netlista za debugovanje
yosys > yosys write_verilog -norename -noexpr -attr2comment out/pfe_debug.v

# Razdvajanje visebitnih mreza
yosys > yosys splitnets -ports -format __v
```

```
# Zamena nedefinisanih vrednosti
yosys > yosys setundef -zero

# Brisanje nepovezanih kola i zica
yosys > yosys clean -purge

# Mapiranje 1 i 0 na odgovarajuće ćelije
yosys > set tech_cell_tiehi {sg13g2_tiehi L_HI}
yosys > set tech_cell_tielo {sg13g2_tielo L_L0}
yosys > yosys hilomap -singleton -hicell {*} $tech_cell_tiehi -locell {*}
    $tech_cell_tielo

# Finalni izveštaji
yosys > yosys tee -q -o "reports/pfe_synth.rpt" check
yosys > yosys tee -q -o "reports/pfe_area.rpt" stat -top pfe_chip {*}
    $liberty_args
yosys > yosys tee -q -o "reports/pfe_area_logic.rpt" stat -top pfe_chip {*}
    $tech_cells_args

# Finalna netlista
yosys > yosys write_verilog -noattr -noexpr -nohex -nodec out/pfe_yosys.v
```

- Zatvoriti program yosys:

```
yosys > exit
```

Ovime smo spremni za fizičku realizaciju dizajna pomoću OpenROAD alata.

8.3 OpenSTA

Sada kada imamo netlistu standardnih ćelija i SRAM-ova, možemo da dobijemo prvi izveštaj o vremenskim karakteristikama. Alat koji se ovde koristi je **OpenSTA**; to je profesionalni mehanizam za statičku analizu vremena (*Static Timing Analysis – STA*) koji je otvorenog koda i podržava većinu Synopsys naredbi za ograničenja dizajna (*Synopsys Design Constraints – SDC*). Podrazumevana ljuska koju koristi zasnovana je na jeziku Tcl. Pomoću nekoliko naredbi možemo da zadamo osnovna vremenska ograničenja, odnosno da alatu objasnimo kako treba da tumači našu netlistu, a zatim možemo generisati izveštaj o vremenskim karakteristikama. Potrebne SDC naredbe su:

- **get_ports <glob pattern>**: prelazi na granicu modula trenutno povezanog dizajna i pokušava da pronađe portove modula koji odgovaraju zadatom obrascu. Postoje i naredbe **get_cells** i **get_nets**, koje funkcionišu na sličan način, ali za ćelije i mreže unutar povezanog dizajna.
- **create_clock -name <name> -period <time in ns> <startpoint>**: definiše jednostavan takt koji potiče iz zadate početne tačke (*startpoint*), obično porta ili izlaza neke ćelije, sa određenim periodom. Ovde zadato ime je ono koje se pojavljuje u izveštajima i koristi se za referenciranje tog takta u narednim komandama. Time se implicitno postavljaju ograničenja za sve putanje koje počinju i završavaju se na flip-flopovima koje pokreće taj takt: signal ne sme da se menja tokom određenog vremena zadržavanja (*hold time*) nakon ivice takta i mora da dostigne svoju konačnu vrednost određeno vreme postavljanja (*setup time*) pre dolaska naredne ivice takta (jedan period kasnije).
- **set_max_delay <time in ns> -from <startpoint>**: predstavlja ograničenje koje zadajemo kolu. Njime definišemo da maksimalno vreme od određene početne tačke do svih

sekvencijalnih krajnjih tačaka (flip-flopova i SRAM-ova) mora biti manje od zadate vrednosti. Imajte u vidu da bez dodatnih argumenata ova naredba potiskuje sva druga ograničenja nad izabranim putanjama, na primer implicitno ograničenje nastalo definisanjem takta.

- **set_input_delay -clock <clock> <delay in ns> <port>**: govori alatu koliko kasno, u odnosu na ivicu takta, ulazni signal čipa može stići na dati port, odnosno koliki se spoljašnji zastoј dodaje, na primer zbog vodova na PCB ploči. Vrednost koju zadate oduzima se od perioda takta, pa unutrašnja logika mora brže da dostigne svoju konačnu vrednost kako bi se uračunalo spoljašnje kašnjenje.
- **set_output_delay -clock <clock> <delay in ns> <port>**: govori alatu koliko vremena moramo da ostavimo na izlazu čipa kako bi sledeći čip imao dovoljno vremena da ispravno uzorkuje signal. Što je ova vrednost veća, manje vremena ostaje za unutrašnju kombinacionu putanju.

Napomena 9:

Obično bismo želeli da zadamo po dva kašnjenja na ulazu i izlazu, korišćenjem argumenata **-min** i **-max**. Razlog za to je što je moguće da signal bude i „prebrz“, odnosno da se promeni suviše rano, dok još uvek treba da ostane stabilan kako bi bio ispunjen uslov zadržavanja (*hold*).

Zadatak 11: Statička analiza

- Pokrenite OpenSTA skriptu:

```
sh > sta scripts/run_sta.tcl
```

- Da li postoji problematicna putanja? Ako postoji, koja je?
- Ako ste radoznali, slobodno malo eksperimentišite sa ograničenjima u alatu **OpenSTA** ili sa naredbama za sintezu u alatu **Yosys**.
- Kraj vežbe.



Svaka čast!